Date: 27/10/2026

C02 - ASSESSMENT TOOL 3

ERROR ANALYSIS

Course Code: DSA03

Course Title: Natural Language Processing

Reg.no: 192425005 **Name:** S. Yuva Guru Mani Varma

---

1. **Biomedical Porter stemming**

- Preprocess by lowercasing and tokenizing before stemming.
- **Porter results:**

Word	Porter stem	Type
infect	infect	base
infected	infect	inflectional
infection	infect	derivational
infectious	infecti	derivational

- **I**nfected can reasonably be conflated with infect; infection and infectious should not automatically be treated as identical because they are different lexical categories and meanings.
- **Better approach:** retain the original token, use lemmatization/POS information for inflections, and use controlled synonym or concept expansion for derivations.

Python program

```
from pathlib import Path
from collections import Counter
import re
from nltk.stem import PorterStemmer

# Change this to where you extracted the dataset.
DATASET_PATH = Path(
    r"C:\Users\sygur\OneDrive\Desktop\NLP-Lab\PubMed_20k_RCT\train.txt"
)

if not DATASET_PATH.exists():
    raise FileNotFoundError(
        f"Dataset file was not found:\n{DATASET_PATH}\n"
        "Download/extract PubMed 20k RCT, then update DATASET_PATH."
    )

stemmer = PorterStemmer()
TOKEN = re.compile(r"[A-Za-z]+")

def tokenize(text):
    return TOKEN.findall(text.lower())
```

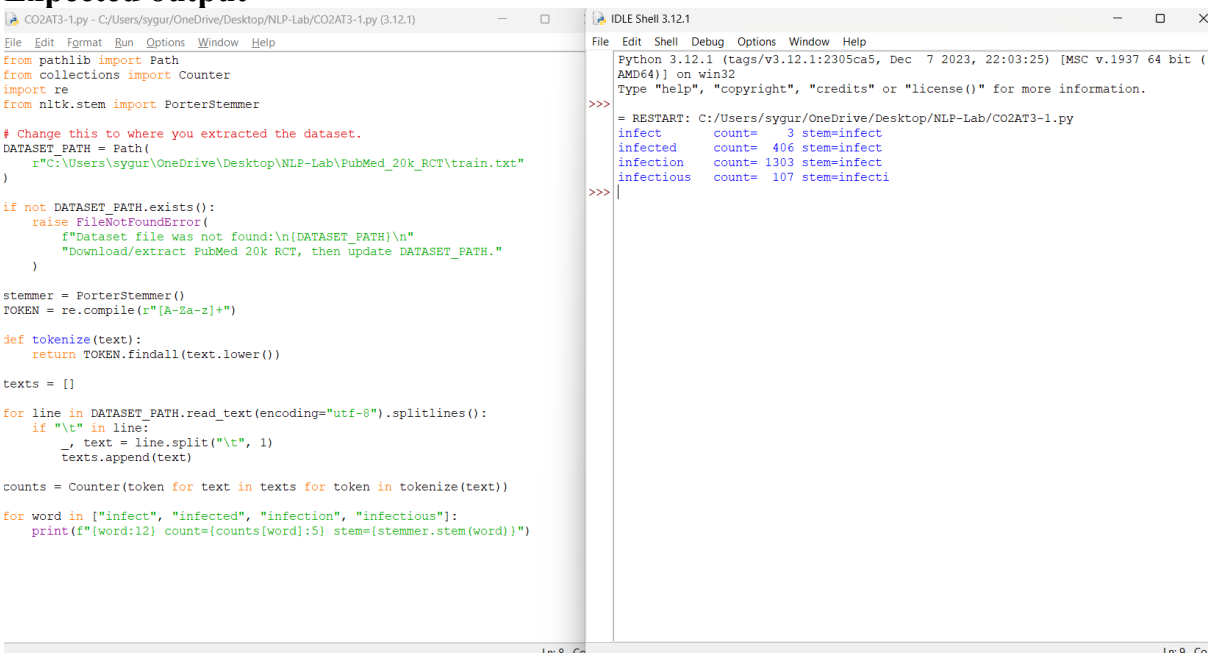
```
texts = []

for line in DATASET_PATH.read_text(encoding="utf-8").splitlines():
    if "\t" in line:
        _, text = line.split("\t", 1)
        texts.append(text)

counts = Counter(token for text in texts for token in tokenize(text))

for word in ["infect", "infected", "infection", "infectious"]:
    print(f"{word:12} count={counts[word]:5} stem={stemmer.stem(word)}")
```

Expected output



2. **Finite-state parser for social media**

- Incorrect analyses occur because a simple parser only handles one affix.
- Correct structures:
    - happiest → happy + est
    - unbelievable → un + believe + able
    - running → run + ing
    - reordering → re + order + ing
    - smartphones → smartphone + s
    - unreadable → un + read + able
- Improved FST sequence:
-

$$\text{START} \rightarrow \text{PREFIX}^* \rightarrow \text{STEM} \rightarrow \text{SUFFIX}^* \rightarrow \text{FINAL}$$

- Include spelling-change rules such as running → run + ing and happiest → happy + est.
- Lookup in a deterministic FST is generally linear in word length, but ambiguity and a larger lexicon increase state/transition complexity.

Python program

```
PREFIXES = ("un", "re")
SUFFIXES = ("able", "est", "ing", "ed", "s")

def analyse_word(word):
    rest = word.lower()
    prefixes = []
    suffixes = []

    # Prefix transition
    if rest.startswith("un") and len(rest) > 3:
        prefixes.append("un")
        rest = rest[2:]
    elif rest.startswith("re") and len(rest) > 3:
        prefixes.append("re")
        rest = rest[2:]

    # Suffix transitions
    while True:
        suffix = next(
            (s for s in SUFFIXES if rest.endswith(s) and len(rest) > len(s) + 1),
            None
        )
        if suffix is None:
            break

        suffixes.insert(0, suffix)
        rest = rest[:-len(suffix)]

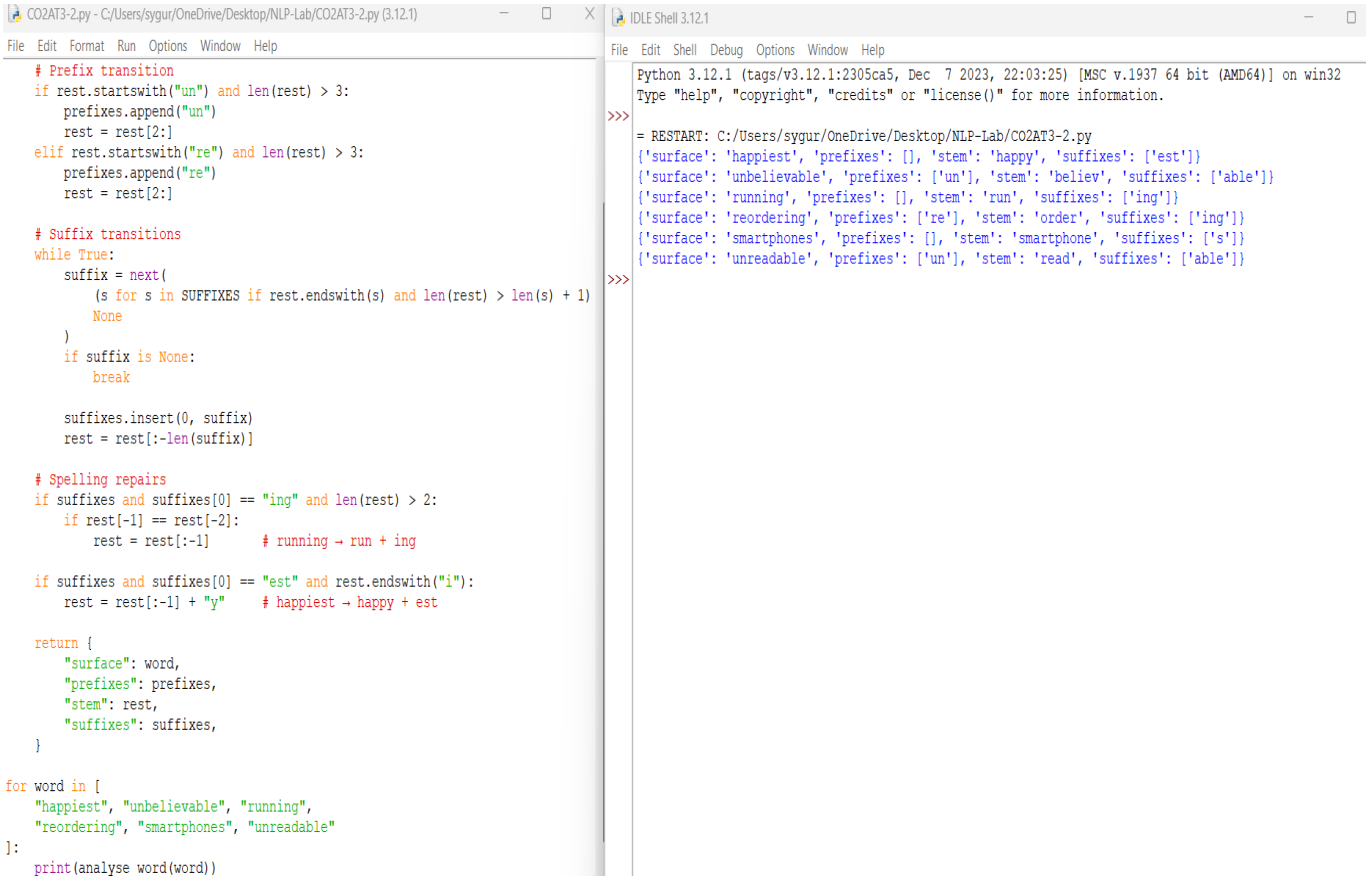
    # Spelling repairs
    if suffixes and suffixes[0] == "ing" and len(rest) > 2:
        if rest[-1] == rest[-2]:
            rest = rest[:-1]       # running → run + ing

    if suffixes and suffixes[0] == "est" and rest.endswith("i"):
        rest = rest[:-1] + "y"     # happiest → happy + est

    return {
        "surface": word,
        "prefixes": prefixes,
        "stem": rest,
        "suffixes": suffixes,
    }

for word in [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]:
    print(analyse_word(word))
```

Expected output



3. Scenario-Based Answer: News Article Classification

Scenario

A news recommendation system stems every word before classification. Technology articles are sometimes predicted as business articles because related but different words are reduced to overly similar stems.

For example:

organization → organ
organizer → organ
organizing → organ
organized → organ

This removes useful distinctions. A technology article discussing an “organizer app” may become too similar to a business article discussing an “organization.”

Solution code

```
import re
import pandas as pd
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
porter = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def tokenize(text):
    return re.findall(r"[A-Za-z]+", str(text).lower())

# Strategy 1: no stemming
def raw_preprocess(text):
    return " ".join(tokenize(text))

# Strategy 2: Porter stemming
def porter_preprocess(text):
    return " ".join(porter.stem(word) for word in tokenize(text))

# Strategy 3: lemmatization
def lemma_preprocess(text):
    return " ".join(lemmatizer.lemmatize(word) for word in tokenize(text))

# BBCNews.csv must contain Text and Category columns
data = pd.read_csv("BBCNews.csv")
data = data.dropna(subset=["Text", "Category"])

X_train, X_test, y_train, y_test = train_test_split(
    data["Text"],
    data["Category"],
    test_size=0.20,
    random_state=42,
    stratify=data["Category"]
)

methods = {
    "Without stemming": raw_preprocess,
    "Porter stemming": porter_preprocess,
    "Lemmatization": lemma_preprocess
}

for method_name, preprocess_function in methods.items():
    model = Pipeline([
        ("vectorizer", TfidfVectorizer(preprocessor=preprocess_function)),
        ("classifier", LogisticRegression(max_iter=1000))
    ])

    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    vocabulary_size = len(model.named_steps["vectorizer"].vocabulary_)
    accuracy = accuracy_score(y_test, predictions)

    print("\nMethod:", method_name)
    print("Vocabulary size:", vocabulary_size)
    print("Accuracy:", round(accuracy, 4))
    print("Confusion matrix:")
    print(confusion_matrix(y_test, predictions))
```

Conclusion

- Without stemming preserves all original information but creates a larger vocabulary.
- Porter stemming reduces vocabulary but may lose semantic differences.
- Lemmatization is usually safer because it produces meaningful base forms.
- For this scenario, POS-aware lemmatization plus preservation of the original token is the best recommendation.

---

4 . Scenario-Based Answer: E-commerce Search

Scenario

A product-search engine uses Porter stemming for all query terms. Customers search for:

watches
watching
washable
washer
washed

The system may incorrectly treat derivational words as equivalent.

washed → wash useful inflectional normalization
washable → washabl adjective; not the same product intent as wash
washer → washer appliance/product; must remain distinct

A customer searching for a "washer" should see washing machines, not cleaning instructions or washable clothing.

Solution code

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

search_terms = [
   "watches",
   "watching",
   "washable",
   "washer",
   "washed"
]

for term in search_terms:
   print(f"{term:12} -> {stemmer.stem(term)}")
```

Improved product-search preprocessing

```
import re
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

def product_search_preprocess(query):
   tokens = re.findall(r"[A-Za-z]+", query.lower())

   # Preserve original query words for exact product matching.
   original_terms = tokens
```

```
    # Conservative lemma terms for recall.
    lemma_terms = [lemmatizer.lemmatize(word) for word in tokens]

    # Porter stems may be used only as lower-priority expansion terms.
    stem_terms = [stemmer.stem(word) for word in tokens]

    return {
        "original_terms": original_terms,
        "lemma_terms": lemma_terms,
        "stem_terms": stem_terms
    }

query = "best washable washer"
result = product_search_preprocess(query)

print("Original terms:", result["original_terms"])
print("Lemma terms:", result["lemma_terms"])
print("Stem terms:", result["stem_terms"])
```

Safer ranking logic

```
def score_product(query_terms, product_title):
    product_words = product_title.lower().split()

    score = 0

    # Strongest: exact word match
    for word in query_terms["original_terms"]:
        if word in product_words:
            score += 3

    # Medium: lemma match
    for word in query_terms["lemma_terms"]:
        if word in product_words:
            score += 2

    # Lowest: stem expansion match
    for word in query_terms["stem_terms"]:
        if word in [stemmer.stem(x) for x in product_words]:
            score += 1

    return score
```

Conclusion

- watches, watching, and washed are mainly inflectional forms.
- washable and washer are derivational forms with distinct meanings.
- Preserve exact words for product intent.
- Use lemmas for controlled recall.
- Do not use aggressive stemming as the main matching method for e-commerce search.